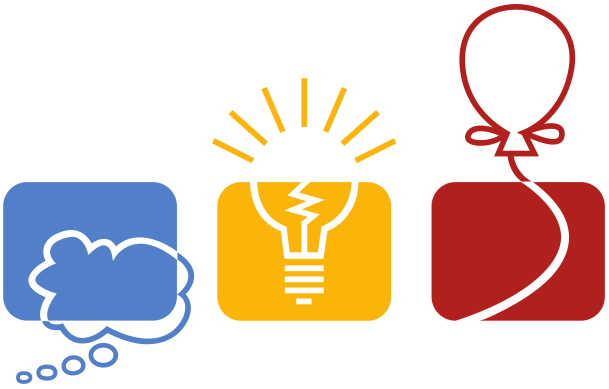
# KCPC Hacking Session 1 Problems
# UKIEPC 2026

## Introduction

**Date:** 21 October 2026
**Contest:** United Kingdom and Ireland Programming Contest 2026
**Problems:** A-M (13 problems)

The 13 UKIEPC 2026 problems cover a broad mix of topics, including letter substitutions, cryptarithms, geometry, ratios, parsing, number theory, graph constructions, dynamic programming, and optimisation. They range from approachable counting and modelling tasks to more involved algorithmic challenges, and are not ordered by difficulty. This sheet accompanies KCPC Year 2 Hacking Session 1 on 21 October 2026; the official contest booklet follows unchanged.

# United Kingdom and Ireland Programming Contest 2026



## Problems


- **A** Austere Sign
- **B** Bust Blasting
- **C** Cryptlet
- **D** Dire Strait
- **E** Elderberry Wine
- **F** Footnotes
- **G** Gloucester Somersault
- **H** Hoptimisation
- **I** Inverse Sum
- **J** Jammer Jam
- **K** Kilt Quilt
- **L** Lawn Stripes
- **M** Mid Clocking


Problems are not ordered by difficulty.
Do not open before the contest has started.

# Problem A
## Austere Sign

We are going to paint a sign but we can only afford stencils for $k$ letters. Update the text of the sign to use only $k$ unique letters without changing the lengths of any of the words, so that we can subtly fix the sign later.

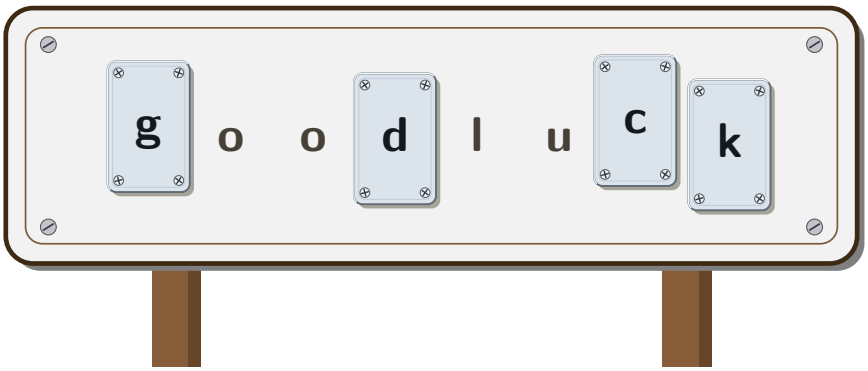



Figure A.1: Illustration of Sample Input 1. You can do this!

### Input

- The first line of input contains the number of words, $n$ ($1 \leq n \leq 1000$), and the number of letters, $k$ ($1 \leq k \leq 26$).
- The second line of input contains $n$ words. Each word consists of between 1 and 20 lowercase latin letters.

### Output

Output a phrase using at most $k$ distinct letters, substituting as few from the original text as possible. If there are multiple possible answers, output any.

| Sample Input 1 | Sample Output 1 |
|---|---|
| 2 3<br>good luck | lool lull |

| Sample Input 2 | Sample Output 2 |
|---|---|
| 3 1<br>this is hard | ssss ss ssss |

| Sample Input 3 | Sample Output 3 |
|---|---|
| 1 6<br>impossible | imbossibee |

# Problem B
## Bust Blasting

Charlie the CNC miller is a cad with a capital 'C'. He is going to clone a valuable statue of a famous voxel-based video game character, and you are going to help him.

The voxel shape is defined by three photos Charlie took, which have been traced into the form of simple, axis-aligned (but potentially concave) polygons representing front ($xy$), side ($zy$), and top ($xz$) views of the statue respectively.

The meaning of "axis-aligned" here is that every edge of every polygon is parallel with one of the $X/Y/Z$ axes.

Any 3-dimensional point not contained by all three polygon projections will be *blasted out of existence* leaving only the true form, which may consist of multiple pieces or no pieces at all.

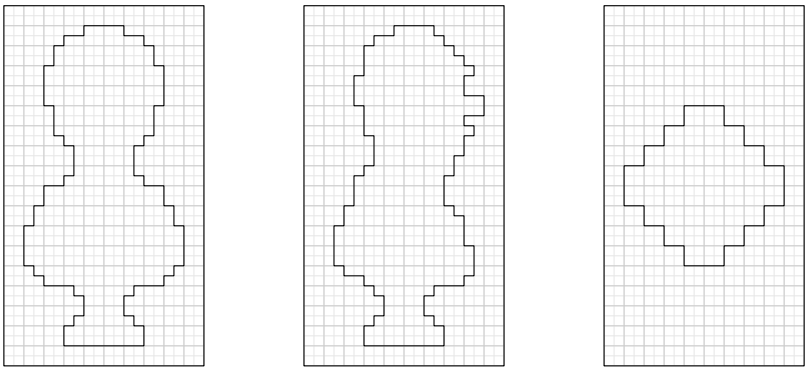


Figure B.1: Illustration of a complex statue.

Calculate the volume of the final statue.

### Input

The input consists of three lines of input, each giving one view as a two-dimensional, axis-aligned polygon in anti-clockwise order, with integer coordinates measured in centimetres.

- First, the number of points, $n_i$ ($4 \leq n \leq 100$).
- Thereafter, $n_i$ pairs of integers $u_{ij}$ and $v_{ij}$ for all $1 \leq j \leq n_i$ ($-10\,000 \leq u, v \leq 10\,000$).

It is guaranteed that for all adjacent vertices $a$ and $b$, $u_{ia} = u_{ib}$ or $v_{ia} = v_{ib}$. No three consecutive vertices are collinear.

### Output

Output the integer total volume of the resulting shape(s) in cubic centimetres.

| Sample Input 1 | Sample Output 1 |
| --- | --- |
| 4 0 0 10 0 10 10 0 10<br>4 5 7 15 7 15 15 5 15<br>4 11 6 11 12 6 12 6 6 | 72 |

# Problem C
## Cryptlet

The 5 Latin letters a-e have each been assigned a distinct digit between 0-9. Given an expression involving the sum of two numbers written as X+Y=Z (each number having the same number of digits, at most 1000, including any leading zeroes), written using letter substitutions, print plausible values for the assignments of all 5 letters in order from a-e as a string of 5 characters.

For instance, given `ab+ac=ca`, you may choose to output `18364`. Or, given `aaa+bbb=eee`, you may output `45789`.

### Input

- The only line of input contains a string of lowercase letters in `[a-e]` and symbols, of the form $X + Y = Z$ and with no spaces, where $1 \leq (|X| = |Y| = |Z|) \leq 1000$.

### Output

If no assignment is possible, output `impossible`.

Otherwise, output any string of 5 distinct digits which gives plausible values for the letters $a$, $b$, $c$, $d$, and $e$ respectively.

| Sample Input 1 | Sample Output 1 |
|---|---|
| ab+ac=ca | 18305 |

| Sample Input 2 | Sample Output 2 |
|---|---|
| aaaa+bbbb=eeee | 34567 |

| Sample Input 3 | Sample Output 3 |
|---|---|
| bba+bbc=bab | 10987 |

# Problem D
## Dire Strait

Mickey has been enjoying sailing on the local lake, but now it's the end of the day and it's time to go back to dock to return the rental boat on time.

Find the shortest path from its current location to the one remaining egress point over water, staying within or passing along the coastline of the lake.

The lake is represented as a simple, potentially concave, polygon having vertices at integer coordinates, and with no 3 consecutive vertices being collinear.

A triangulation of the lake into $(n-2)$ pieces is also provided.

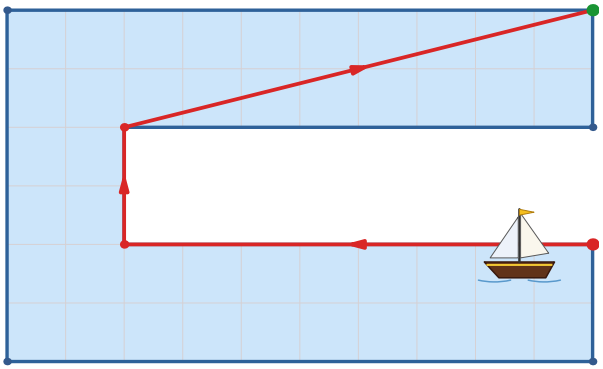


Figure D.1: Illustration of Sample Input 1.

### Input

- The number of vertices in the polygon representing the lake, $n$ ($3 \leq n \leq 100\,000$), and the indices of the two vertices between which to travel, $a$ and $b$ ($1 \leq a, b \leq n; a \neq b$).
- $n$ further lines, each giving integer coordinates $x_i$ $y_i$ of the $i$th vertex of the polygon ($-10^9 \leq x, y \leq 10^9$) in anti-clockwise order.
- $(n-2)$ further lines giving a triangulation of the lake. Each line specifies the 1-based indices of three vertices $u$ $v$ $w$ (in anti-clockwise order).

The given triangles correspond to the polygon, thus cover the lake without any self-overlaps, holes, or other hazards.

### Output

Output the shortest distance from point $a$ to point $b$ without exiting the lake.

The output must be accurate to an absolute or relative error of $10^{-6}$.

**Sample Input 1**

```
8 3 7
0 0
10 0
10 2
2 2
2 4
10 4
10 6
0 6
1 2 3
1 3 4
1 4 8
4 5 8
5 7 8
5 6 7
```

**Sample Output 1**

```
18.246211251
```

**Sample Input 2**

```
14 1 9
-3 -1
-1 -1
-1 -3
1 -3
1 -1
3 -1
3 1
1 1
1 3
5 3
5 5
-1 5
-1 1
-3 1
1 2 14
6 8 13
3 4 5
9 12 13
8 9 13
6 13 14
2 5 14
5 6 14
10 11 12
2 3 5
9 10 12
6 7 8
```

**Sample Output 2**

```
5.6568542495
```

# Problem E
## Elderberry Wine

Jeanne and her son Semi-Demi-John are engaged in the elderberry-wine making business.

The recipe requires, aside from a closely-guarded secret quantity of the letter $r$, strict proportions of 12 parts fresh elderberries, 12 parts brewing sugar, and 40 parts water. As long as one or more ingredients are lacking, the team will place an order for as much extra as needed to make up the weight, and start again.

For instance:

- If they have 18kg of elderberries, 9kg of sugar, and 100kg of water, they can use 9kg of elderberries, all 9kg of the sugar, and 30kg of the water and then they are stuck.
- They must then order 9kg more of sugar in order to proceed and use 9kg, 9kg, and 30kg again.
- After this they must order 12kg each of elderberries and sugar in order to finally use 12kg, 12kg, and 40kg of elderberries, sugar, and water respectively.

| **Phase** | **Eb (kg)** | **Su (kg)** | $H_2O$ **(kg)** |
|---|---|---|---|
| Initial Stock | 18 | 9 | 100 |
| First Brewing Run | -9 | -9 | -30 |
| Remaining Stock (Stuck) | 9 | 0 | 70 |
| Sugar Order | 0 | +9 | 0 |
| Stock After First Order | 9 | 9 | 70 |
| Second Brewing Run | -9 | -9 | -30 |
| Remaining Stock (Stuck) | 0 | 0 | 40 |
| Berry + Sugar Order | +12 | +12 | 0 |
| Stock After Second Order | 12 | 12 | 40 |
| Final Brewing Run | -12 | -12 | -40 |

## Input

The input consists of one line containing three floating-point numbers $a$, $b$, $c$ ($0 \leq a, b, c \leq 10^{16}$): the number of kilograms of elderberries, sugar, and water respectively.

## Output

Output three floating-point numbers $a'$ $b'$ $c'$ giving the cumulative total quantities of elderberries, sugar, and water that will be ordered respectively.

The output must be accurate to an absolute or relative error of $10^{-6}$.

| **Sample Input 1** | **Sample Output 1** |
|---|---|
| `18 9 100` | `12 21 0` |

| Sample Input 2 | Sample Output 2 |
| --- | --- |
| 1.5 17 5 | 15.5 0 51.666667 |

| Sample Input 3 | Sample Output 3 |
| --- | --- |
| 3 4 7000000000000000 | 2099999999999997 2099999999999996 0 |

# Problem F
## Footnotes

Your colleague Cynthia likes writing asides in messages (but then again, don't we all (and there's nothing strictly wrong with that anyway)). As some people find the text hard to read this way, we are going to try to extract some parts of the sentences out into equally-sized footnotes instead of brackets.

For example, `this(complicated(nested(passage)))` could become:

```
this[1]
[1] complicated[2]
[2] nested(passage)
```

Given one such message, an unbroken string of up to $10^6$ characters (letters and brackets, without any spaces), isolate exactly two-thirds of the Latin letters into footnotes, each footnote containing exactly the same number of Latin letters.

### Input

- One line containing a string $s$ ($1 \leq |s| \leq 10^6$), each character in $s$ being a lowercase letter or an opening/closing round bracket.

It is guaranteed that the brackets in $s$ are properly matched and that the content of a pair of brackets is always non-empty.

### Output

Output three lines in the form shown above, using exactly two footnotes in square brackets to indicate where passages were relocated. If there is no way of doing this, output `impossible` instead.

Your answer will be tested by substituting the footnotes back (inside brackets) and comparing character-by-character with the original text.

| **Sample Input 1** | **Sample Output 1** |
|---|---|
| `(a(b(c(d(e(f(g(h(i)))))))))` | `(a(b(c[1])))`<br>`[1] d(e(f[2]))`<br>`[2] g(h(i))` |

| **Sample Input 2** | **Sample Output 2** |
|---|---|
| `a(b)(b)(b)((cc))` | `impossible` |

| **Sample Input 3** | **Sample Output 3** |
|---|---|
| `a((b)(b))(b)((cc))` | `a[1](b)[2]`<br>`[1] (b)(b)`<br>`[2] (cc)` |

# Problem G
## Gloucester Somersault

A piece of cheese with polygonal cross-sectional area is rolling along a flat surface as part of a major sporting event.

The piece of cheese rotates clockwise continuously, pivoting around the vertex with lowest $y$-coordinate and, in case of a tie, greatest x-coordinate.

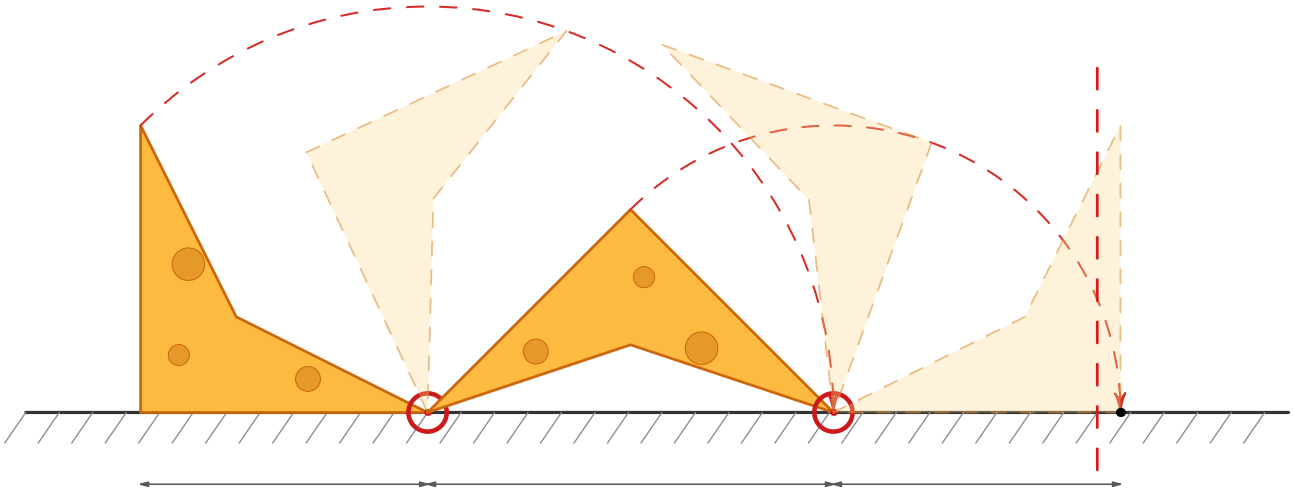


Figure G.1: Illustration of Sample Input 1.

The finish line is $m$ metres from the origin point and must be crossed at ground level, meaning that one of the vertices on the ground is strictly past the line.

How many revolutions does the piece of cheese make in order for at least one part touching the ground to be strictly past the line?

## Input

- The first line of input contains the number of vertices of the cheese piece, $n$ ($3 \leq n \leq 10\,000$), and the integer $x$-coordinate of the finish line in metres, $m$ ($1 \leq m \leq 10^6$).
- The next $n$ lines of input contain the $x$ and $y$ coordinates of cheese vertices, in anti-clockwise order, $x_i$ and $y_i$ ($0 \leq x_i, y_i \leq 10\,000$).

No three adjacent vertices on the shape are collinear and at least one vertex has $y = 0$. The cheese has not yet crossed the line.

## Output

Output the number of full 360-degree rotations performed by the piece of cheese until it strictly crosses the line on the ground, rounded up to an integer.

**Sample Input 1**

```
4 10
0 0
3 0
1 1
0 3
```

**Sample Output 1**

```
1
```

**Sample Input 2**

```
4 100
51 0
53 2
51 4
49 2
```

**Sample Output 2**

```
5
```

# Problem H
## Hoptimisation

You are practicing a sequence of hops on a rather boring hopscotch track consisting of several consecutive cells in a straight line, spaced 1m apart.

You have a planned route of cell names to visit. The total hopping distance depends on how you assign these names to the physical positions (1 to $k$).

For example, to complete the route A, B, A, C with the layout [A, B, C, D, E], the hops take $1 + 1 + 2 = 4$ metres total.




Figure H.1: Illustration of Sample Input 1.

Given your sequence of hops, determine how to arrange the names across the cells (left to right) to minimize the total hopping distance.

### Input

- One line of input containing the number of cells, $k$ ($1 \leq 16$), and the number of hops, $n$ ($1 \leq n \leq 10^6$).
- A second line of input containing $n$ lower-case strings, the $i$ th string $s_i$ containing the target cell for the $i$ th hop. ($1 \leq |s| \leq 10$).

It is guaranteed that no more than $k$ distinct cell names are given.

### Output

Output $k$ strings of up to 10 lower-case letters each: the names you want to assign to each cell from left to right. If there are multiple potential solutions, you may output any.

In case fewer than $k$ names are given in the input, feel free to invent more to fill the space.

| Sample Input 1 | Sample Output 1 |
|---|---|
| 5 4<br>hop skip hop jump | anything jump hop skip something |

| Sample Input 2 | Sample Output 2 |
|---|---|
| 9 16<br>a b c a b c a x y z x y z e f g | g f e z y x a c b |

# Problem I
## Inverse Sum

Given an integer $n > 1$, and the equation:

$$\frac{1}{n} = \frac{1}{a} + \frac{1}{b}$$

Find two distinct positive integers $a, b$ which both satisfy the above equation, and have a minimal sum $a + b$.

### Input

- The only line of input consists of one integer $n$ ($1 < n \leq 10^8$) — the number for the left hand side of the equation.

### Output

Output the best choices for $a$ and $b$ in ascending order.

| Sample Input 1 | Sample Output 1 |
|---|---|
| 4 | 6 12 |

| Sample Input 2 | Sample Output 2 |
|---|---|
| 107 | 108 11556 |

| Sample Input 3 | Sample Output 3 |
|---|---|
| 999999 | 1998000 2002000 |

| Sample Input 4 | Sample Output 4 |
|---|---|
| 1000000 | 1800000 2250000 |

# Problem J
## Jammer Jam

Swimming for exercise involves assigning swimmers to lanes suited to their fitness level.

We know the velocity of each swimmer in metres per hour. The swimmers start in any order but inevitably they will eventually become stuck behind the slowest swimmer in the lane and start matching their pace.

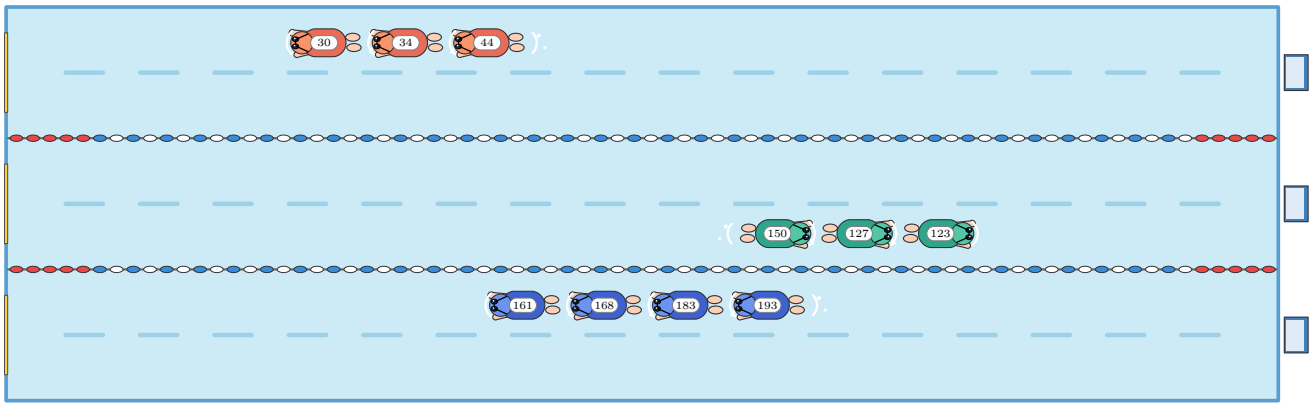



Figure J.1: Illustration of Sample Input 3.

If we can assign swimmers to $l$ lanes, how can we ensure that the average speed of all of the swimmers in the pool (measured in metres per hour) is as high as possible? Output this average speed and an assignment of swimmers to lanes.

### Input

- One line containing the number of swimmers $n$ and the number of lanes $l$ ($1 \leq l \leq n \leq 3\,000$).
- One line containing $n$ integers giving speeds $a_1 \ldots a_n$ of the swimmers, in metres per hour ($1 \leq a \leq 10^6$).

### Output

Output the maximum achievable average speed (mean metres per hour) on one line, followed by a possible assignment of swimmers to lanes to achieve this on the next line. The $i$th integer on this line is the lane number for the $i$th swimmer, $1 \leq a \leq l$.

The output must be accurate to an absolute or relative error of $10^{-6}$.

| Sample Input 1 | Sample Output 1 |
|---|---|
| 3 2<br>100 200 50 | 100<br>1 2 1 |

**Sample Input 2**

```
4 2
11 22 34 47
```

**Sample Output 2**

```
22.5
1 1 2 2
```

**Sample Input 3**

```
10 3
34 123 127 161 168 30 44 150 183 193
```

**Sample Output 3**

```
110.3
1 2 2 3 3 1 1 2 3 3
```

# Problem K
## Kilt Quilt

An image set to be painted onto a tartan canvas consists of $n$ rows of $m$ pixels with each colour represented as a lowercase letter of the alphabet.

You are going to paint the image using long brush strokes that cover either a whole row of $m$ pixels, or a whole column of $n$ pixels. Each brush stroke overwrites any pixels that were previously there. You must cover the whole canvas with brush strokes.

Is it possible to do so? If so, output a sequence of instructions of the form `x=[1-m] [a-z]` or `y=[1-n] [a-z]`. If there are multiple sequences, output any of the shortest possible lists of instructions.

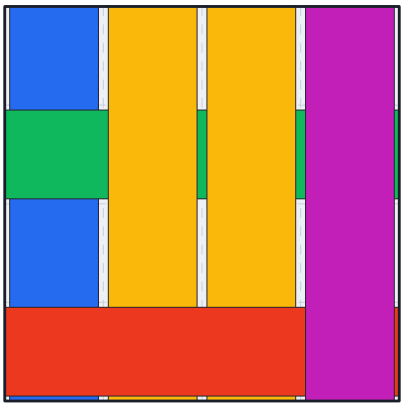


Figure K.1: Illustration of Sample Input 2.

### Input

- The first line of input contains $n$ and $m$ ($1 \leq n, m \leq 200$)
- The next $n$ lines of input contain a string of $m$ lowercase characters in `a-z`, each representing a colour.

### Output

On the first line, output the minimum number of instructions needed. If no sequence of instructions covers the canvas, output `impossible` and exit.

On each of the next lines, output one instruction to follow in order. For example, "`x=5 z`", and "`y=100 b`" are both valid instructions.

| Sample Input 1 | Sample Output 1 |
|---|---|
| 4 6<br>aaaaba<br>aaaaba<br>aaaaba<br>aaaaba | 5<br>y=1 a<br>y=2 a<br>y=3 a<br>y=4 a<br>x=5 b |

| Sample Input 2 | Sample Output 2 |
| --- | --- |
| 4 4<br>laaq<br>daaq<br>laaq<br>bbbq | 6<br>x=1 l<br>y=2 d<br>x=2 a<br>x=3 a<br>y=4 b<br>x=4 q |

# Problem L
## Lawn Stripes

Lawn stripes are drawn using a roller to flatten grass blades in a particular direction and impart a sheen that, at a distance, is distinguishable from blades flattened in another direction. This is typically for aesthetic appeal.

We are going to draw lawn stripes in our square garden using a roller in any of the 4 cardinal directions. The roller must stay on the lawn throughout. The final direction of a piece of lawn is determined by the last time the roller ran over it, including starting from or finishing there.

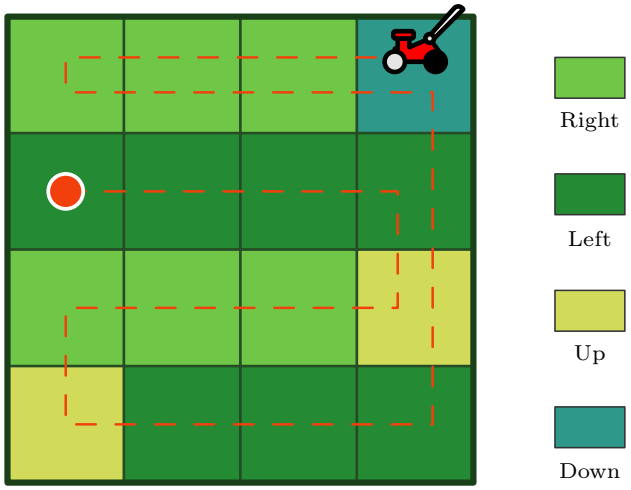



Figure L.1: Illustration of Sample Input 1.

Given a description of the lawn and the starting point of the roller, determine how to draw the pattern.

### Input

- The first line of input contains the width and depth in metres of the lawn, $n$ ($1 \leq n \leq 100$)
- The second line of input contains the row and column of the roller, $r$ and $c$ ($1 \leq r, c \leq n$).
- Each of the next $n$ lines contains $n$ characters from `(l|r|u|d)` giving the desired direction of a square metre of lawn.

### Output

If it is not possible to draw the pattern, output `impossible`. Otherwise, demonstrate a solution in under $10n^2$ moves by outputting the number of moves to use on one line, followed by the moves on the remaining lines.

Each move consists of a distance in metres $d$ and a direction from `(l|r|u|d)`. A roller following the instructions must roll over every square metre without leaving the lawn.

| Sample Input 1 | Sample Output 1 |
| --- | --- |
| 4<br>1 4<br>rrrd<br>llll<br>rrru<br>ulll | 8<br>3 l<br>3 r<br>3 d<br>3 l<br>1 u<br>3 r<br>1 u<br>3 l |

| Sample Input 2 | Sample Output 2 |
| --- | --- |
| 2<br>1 1<br>rd<br>ul | impossible |

# Problem M
## Mid Clocking

We have a number of processor slots in a row. The processors are each dedicated to a single task that they can complete before going into a busy-looping mode while they wait for the next instruction.

A processor can run at 100% speed if it has enough cooling, and that requires there be no adjacent processor. If a processor has 1 adjacent processor, it only runs at 60% of maximum speed. If it has 2 adjacent processors it can only run at 30% of maximum speed.

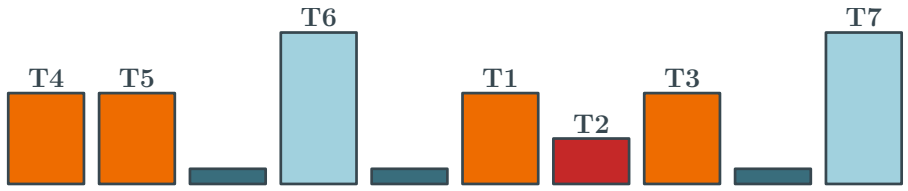



Figure M.1: Illustration of Sample Input 2.

We know how many seconds each task will take running on a processor at full speed, starting from $t = 0$, and need to allocate tasks to processors to ensure the last task finishes as soon as possible.

### Input

- The number of tasks to run, $n$, and the number of processor slots, $s$ ($1 \leq n \leq s \leq 10^6$).
- $n$ further integers giving the duration of each task $t_i$ in seconds if performed at 100% speed ($1 \leq t_i \leq 10^9$).

### Output

Output the minimum time by which all of the tasks can be finished.

The output must be accurate to an absolute or relative error of $10^{-6}$.

| Sample Input 1 | Sample Output 1 |
|---|---|
| 4 5<br>1 1000 1 1 | 1000 |

| Sample Input 2 | Sample Output 2 |
|---|---|
| 7 10<br>5 2 3 5 4 6 7 | 8.33333333 |